

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Съдържание:

1. Elixir - рекурсия
2. Loops through recursion (Цикли чрез рекурсия)
3. “Reduce” и “map” алгоритми
4. Enumerables
5. Eager vs Lazy
6. Pipe оператор
7. Stream
8. IO module
9. File module
10. Path module
11. Процеси и ръководители на групи
12. iodata and chardata
13. Работа с файлове

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

1. Elixir - Рекурсия

Рекурсията е основен принцип при функционалните езици за програмиране, включително Elixir. Вместо да се използват цикли, Elixir предпочита рекурсивни функции за итерация.

Примери за рекурсивни функции

1) Пример: Изчисляване на факториел на число

Рекурсивен подход за изчисляване на факториел на дадено число.

```
defmodule Math do
  def factorial(0), do: 1
  def factorial(n) when n > 0 do
    n * factorial(n - 1)
  end
end
```

```
IO.puts(Math.factorial(5)) # Извежда 120
```

Анализ

`factorial(0)` е *базов случай*, който спира рекурсията.

`factorial(n)` извиква сама себе си, с параметър `n - 1`.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

2) Пример: Числа на Фибоначи

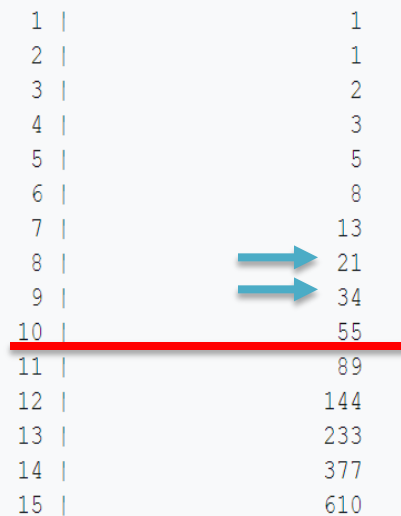
В последователността на Фибоначи всяко число е сумата на двете предходни числа. Последователността започва с 0 и 1, и след това продължава: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 и т.н.

```
defmodule Fibonacci do
  # Базовите случаи
  def fib(0), do: 0
  def fib(1), do: 1

  # Рекурсивната функция
  def fib(n) when n > 1 do
    fib(n - 1) + fib(n - 2)
  end
end

# Пример за извикване на функцията
n = 10
IO.puts("Fibonacci номер #{n} е #{Fibonacci.fib(n)}")
# Извежда: Fibonacci номер 10 е 55
```

фиг.1



1	1
2	1
3	2
4	3
5	5
6	8
7	13
8	21
9	34
10	55
11	89
12	144
13	233
14	377
15	610

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Анализ

Базови случаи:

$\text{fib}(0)$ връща 0, тъй като първото число в последователността на Фибоначи е 0.

$\text{fib}(1)$ връща 1, тъй като второто число е 1.

Рекурсивен случай:

Функцията $\text{fib}(n)$ за $n > 1$ извиква себе си два пъти: $\text{fib}(n - 1)$ и $\text{fib}(n - 2)$. Това е основната рекурсивна идея за изчисляване на стойността от предходните два термина. Като резултата е извеждане на 10-ото число от последователността на Фибоначи.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

3) Пример: Обратно извеждане на списък

Може да се напише и рекурсивна функция, която връща списък в обратен ред.

```
defmodule ListUtils do
  def reverse([], do: [])
  def reverse([head | tail]) do
    reverse(tail) ++ [head]
  end
end
```

```
IO.inspect(ListUtils.reverse([1, 2, 3, 4, 5]))
# Извежда [5, 4, 3, 2, 1]
```

Тук функцията reverse/1 взема списък и рекурсивно извиква себе си, за да обработи остатъка от списъка, след което добавя главата в края. Обаче, този подход е неефективен, защото при всяко извикване на ++ се създава нов списък (Това вече беше споменато в предишни лекции).

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Защо е неефективен обаче този подход?

Подходът, използван в примера за обръщане на списък, е неефективен, защото операторът ++, който се използва за свързване на списъци, е с линейно време ($O(n)$), тъй като трябва да обходи целия списък от лявата страна. Така във всяко рекурсивно извикване се създава нов списък, което ще доведе до излишни операции и увеличаване на времевата сложност до $O(n^2)$. Но ние можем да оптимизираме функцията, като използваме вторичен списък, който служи като акумулатор. По този начин можем да избегнем многократното свързване на списъци и да постигнем линейна сложност $O(n)$.

По-оптимизираната версия на функцията reverse може да се представи по следния начин:

Оптимизиран вариант

```
defmodule ListUtils do
  def reverse(list), do: reverse(list, [])

  defp reverse([], acc), do: acc
  defp reverse([head | tail], acc) do
    reverse(tail, [head | acc]) # Добавяме главата в акумулатора
  end
end

IO.inspect(ListUtils.reverse([1, 2, 3, 4, 5])) # Извежда [5, 4, 3, 2, 1]
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Анализ

- **Акумулатор:** Добавяме втори аргумент към рекурсивната функция, който служи за акумулатор (асс). Първоначално асс е просто празен списък.
- **Обратно добавяне:** Вместо да добавяме елементите в края на новия списък с ++, добавяме head към асс, което е много по-ефективно. Операторът | (конструктор за списък) добавя елемента в началото на списъка, което е операция с константно време $O(1)$.
- **Базов случай:** Когато оригиналният списък е празен ([]), връщахме акумулатора, който вече съдържа елементите в обратен ред.

Този подход е значително по-ефективен и позволява на функцията да работи с линейна сложност $O(n)$, вместо време повдигнато на квадрат $O(n^2)$.

Или сравнявайки двата варианта:

Първи вариант: reverse([]) и reverse([head | tail])

```
defmodule ListUtils do
  def reverse([]), do: []
  def reverse([head | tail]) do
    reverse(tail) ++ [head]
  end
end
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Как работи?

Функцията reverse/1 рекурсивно извиква себе си, докато не достигне базовия случай (т.е. празен списък). Когато обходи списъка, за всеки елемент, използва оператора ++, за да добави текущата head в края на новия списък.

Каква е времевата сложност:

Основният недостатък тук е, че операторът ++ за свързване на списъци е $O(n)$ сложност спрямо дължината на списъка отляво. В общия случай, за списък от дължина N , извикванията на reverse ще доведат до $O(n^2)$ сложност, защото за всеки елемент head, списъкът трябва да се обходи отново, за да добави head в края на новия списък.

Втори вариант: Използване на акумулатор за обръщане на списъка

```
defmodule ListUtils do
  def reverse(list), do: reverse(list, [])

  defp reverse([], acc), do: acc
  defp reverse([head | tail], acc) do
    reverse(tail, [head | acc])
  end
end
```

Добавяме главата в акумулатора

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Как работи?

В този вариант функцията използва втори аргумент (асс), който функционира като акумулатор. Когато head е добавен в асс, той постоянно се увеличава, а новият списък остава в обратен ред, без да е необходимо да се обхождат елементите, за да се добавят отново.

Каква е времевата сложност?

Времевата сложност на този подход е $O(n)$, тъй като всеки елемент се добавя в акумулатора само веднъж и операцията с конструктор на списъци (!) е $O(1)$.

Този метод е значително по-ефективен, особено за дълги списъци.

Сравнение по отношение на – времева сложност, използвана памет и четимост на кода

Каква е времевата сложност?

- Първият вариант: $O(n^2)$ поради необходимостта от свързване на списъци.
- Вторият вариант: $O(n)$ благодарение на акумулатора.

Памет:

- В първия вариант всеки етап от рекурсията създава частичен списък, което води до увеличаване на паметта, използвана за съхранение на междинни резултати. Рекурсивната дълбочина ще се увеличи, докато не достигне базовия случай, и всеки нов списък запазва историята на предишните елементи.
- Вторият вариант използва паметта по-икономично, тъй като се съхранява само текущото състояние на акумулатора, без излишни данни.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Четимост:

Четимостта на двата кода е приблизително равна, но вторият вариант е малко по-добре структуриран, като разделя логиката на публична и частна рекурсивна функции.

Обобщение

Като цяло, вторият вариант с акумулатор е значително по-ефективен и се препоръчва при работа с по-дълги списъци или при необходимост от обръщане на списъци в производствени среда. Първият вариант е подходящ само за много малки списъци, поради своята неефективност.

Забележка:

Тъй като в примерите е използван символът `->` ще направим кратко обяснение. В Elixir обикновено този символ се нарича "функционален оператор" или "оператор за стрелка" (arrow operator). В контекста на езиците за програмиране, се среща като "оператор за показване на резултата" или "оператор за стрелка, използван за съвпадение". В Elixir се използва за обозначаване на функция или условие в конструкции като `case`, `cond` и `fn`. Например:

```
case value do
  :ok -> "Operation was successful"
  :error -> "An error occurred"
end
```

В този контекст, `->` разделя шаблона (лявата страна) от действието или резултата (дясната страна), което да се извърши при съвпадение. При дефиниране на анонимни функции, `->` се използва по следния начин: `my_function = fn x -> x + 1 end`

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

4) Пример: Генериране на комбинации

Може да се генерират всички комбинации от даден набор от елементи.

```
defmodule Combinations do
  def combine([], _), do: [[]]
  def combine([head | tail], k) do
    with_head = combine(tail, k - 1) |> Enum.map(fn combo -> [head | combo] end)
    without_head = combine(tail, k)
    with_head ++ without_head
  end
end
```

```
IO.inspect(Combinations.combine([1, 2, 3], 2))
# Извежда комбинации от елементи
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

5) Пример: Факториел с мемоизация

Какво е мемоизация?

Мемоизацията е подход за оптимизиране на рекурсивни функции, при които се запазват резултатите от предишни изчисления, така че когато функцията бъде извикана отново с идентични аргументи, резултатът може да бъде върнат директно без повторно изчисление.

Как работи мемоизацията?

Мемоизацията използва структура от данни (обикновено хеш таблица или асоциативен масив), за да запази резултатите от функцията. Когато функцията е извикана, първо се проверява дали резултатът за конкретния набор от аргументи вече е изчислен:

- **Проверка:** Когато функцията се извика с определени аргументи, тя проверява дали резултатът от предишно извикване съществува в кеша.
- **Връщане на кеширания резултат:** Ако кешираният резултат е наличен, функцията връща него вместо да извършва изчислението отново.
- **Изчисление и кеширане:** Ако кешираният резултат не е наличен, функцията изчислява резултата, запазва го в кеша и го връща.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Основни стъпки за създаване на Факториел с мемоизация:

1. Основна функция (factorial/1): Функцията трябва да проверява дали числото е отрицателно и да извика вътрешната функция с начален кеш (празен map).
2. Вътрешна функция (factorial/2):
 - Трябва да проверява дали има кеширано значение за n .
 - Ако няма се изчислява факториела, чрез рекурсивната функция за $n - 1$ и се актуализира кеша.
 - Ако има кеширано значение, просто се връща резултата.

Така, при последващи извиквания на функцията за вече пресметнати стойности, ще се възползва резултата от кеша и ще избегне повторното изчисление.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
defmodule Memoization do
```

```
# Дефинираме публична функция, която изчислява факториел от n.
```

```
# Проверяваме дали n е отрицателно, в такъв случай връщаме грешка.
```

```
def factorial(n) when n < 0, do: {:error, "Negative numbers are not allowed"}
```

```
# Дефинираме публична функция за факториел, която приема n и кеш (празен мап в началото).
```

```
def factorial(n), do: factorial(n, %{})
```

```
# Дефинираме вътрешна (private) функция, която е основната функция за изчисляване на факториела.
```

```
# Тя приема n и кеш, и обработва случая, когато n е 0.
```

```
# Факториел от 0 е 1, и кешираме стойността.
```

```
defp factorial(0, cache), do: {1, Map.put(cache, 0, 1)}
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
# Дефинираме и вътрешна (private) функция, която обработва случаи с  $n > 0$ .
defp factorial(n, cache) do
  # Проверяваме дали вече имаме кеширана стойност за n.
  case Map.get(cache, n) do
    nil ->
      # Ако няма кеширана стойност, рекурсивно изчисляваме факториела за  $n - 1$ 
      {result, updated_cache} = factorial(n - 1, cache)

      # Изчисляваме факториела за n като  $n * \text{факториела на } n - 1$ .
      factorial_result = n * result

      # Връщаме резултата и актуализирания кеш.
      {factorial_result, Map.put(updated_cache, n, factorial_result)}

    # Ако имаме кеширана стойност, я връщаме.
    cached_result ->
      {cached_result, cache}
  end
end
end
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
# Пример за използване на функцията  
{result, _cache} = Memoization.factorial(5)  
IO.puts("Factorial of 5 is #{result}")  
# Изход: Factorial of 5 is 120
```

```
# Пример за повторно извикване на функцията, използвайки вече кеширан резултат.  
{result, _cache} = Memoization.factorial(5)  
IO.puts("Factorial of 5 is #{result}")  
# Изход: Factorial of 5 is 120 (извикването е по-бързо)
```


Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
defmodule Memoization do
  def factorial(n) when n < 0, do: {:error, "Negative numbers are not allowed"}
  def factorial(n), do: factorial(n, %{})
  # Основната функция, която използва втори аргумент за кеширане
  defp factorial(0, cache), do: {1, Map.put(cache, 0, 1)}
  defp factorial(n, cache) do
    case Map.get(cache, n) do
      nil ->
        {result, updated_cache} = factorial(n - 1, cache)
        factorial_result = n * result
        {factorial_result, Map.put(updated_cache, n, factorial_result)}
      cached_result ->
        {cached_result, cache}
    end
  end
end

# Използване на функцията
{result, _cache} = Memoization.factorial(5)
IO.puts("Factorial of 5 is #{result}") # Изход: Factorial of 5 is 120
{result, _cache} = Memoization.factorial(5)
IO.puts("Factorial of 5 is #{result}") # Изход: Factorial of 5 is 120 (извикването е по-бързо)
Factorial of 5 is 120
Factorial of 5 is 120
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Предимства на мемоизацията

- **Увеличава производителността:** Значително намалява времето за изпълнение на рекурсивни функции, като елиминира многократните изчисления на стойности, които вече са били изчислени.
- **Улеснява работата със сложни функции:** Позволява по-лесно разделяне на проблема на подпроблеми чрез запамятаване на резултати.
- **Улеснява управлението на ресурси:** След като резултатите са кеширани, функцията става по-ефективна и използва по-малко ресурси.

Недостатъци на мемоизацията

- **Употреба на паметта:** Мемоизацията може да изисква значително количество памет, особено ако функциите имат много уникални аргументи и резултати.
- **Сложност при реализация:** Възможно е да усложни кодирането, особено ако не е необходимо или не е желателно запамятаването на резултати за определени функции.

В заключение, мемоизацията е мощен инструмент за оптимизация, който може да подобри производителността на рекурсивните функции и алгоритми за динамично програмиране, но трябва да се прилага внимателно, за да се избегне ненужната употреба на памет.

Рекурсия и проблеми с паметта

Рекурсивните функции без правилно управление на базовите случаи и без оптимизация (като хвърляне на изключения за прекратяване на рекурсията) могат да доведат до проблеми с паметта, поради дълбочината на стека.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Пример, който може да доведе до препълване на паметта

```
defmodule InfiniteRecursion do
  def count_down(n) do
    IO.puts(n)
    count_down(n - 1)
  # Без базов случай, което води до
  # безкрайна рекурсия
  end
end

# Извикване, ще доведе до
# препълване на стека
# InfiniteRecursion.count_down(5)
```

```
defmodule InfiniteRecursion do
  def count_down(n) do
    IO.puts(n)
    count_down(n - 1)
  # Без базов случай, което води до безкрайна рекурсия
  end
end

# Извикване, ще доведе до препълване на стека
InfiniteRecursion.count_down(5)
```

```
-1078
-1079
-1080
-1081
-1082
-1083
-1084
-1085
-1086
-1087
-1088
-1089
-1090
-1091
-1092
-1093
-1094
-1095
-1096
-1097
-1098
-1099
-1100
-1101
-1102
-1103
-1104
-1105
-1106
-1107
-1108
-1109
-1110
-1111
-1112
-1113
-1114
-1115
```

Анализ

В примера функцията `count_down/1` не съдържа базов случай и ще продължи да извиква себе си, докато стекът не се препълни. Ако активирате това извикване (например `InfiniteRecursion.count_down(5)`), ще получите `System.stacktrace not evaluated`, което е индикатор за препълване на стека.

Обобщение

Този пример показва как можете да реализирате рекурсивна логика в Elixir, за да изчислите Факториел от дадено число. Както при варианта на факториел без мемоиззация, така и тук при нейното прилагане е важно да се внимава с дълбочината на рекурсията, тъй като дълбоката рекурсия може да доведе до проблеми с паметта (stack overflow).

2. Цикли чрез рекурсия

В Elixir, традиционните цикли `while` и `for`, каквито има в езиците от императивен стил (като C или Python), не се използват. Вместо тях в Elixir се прилага рекурсия и функции от модула `Enum` за итерации над колекции. Решенията, обикновено, се реализират с рекурсия, или чрез вградени функции за работа с колекции. Т.е. вместо да се използват цикли (`for`, `while` и т.н.), в Elixir може да се извършват итерации, като се използват рекурсивни функции. Например, може да се създадва функция, която обхожда списък и извежда елементите му.

2.1. Пример с рекурсия

Пример 1 с обхождане и сумиране на елементи в списък

```
defmodule MyList do
  def sum([], do: 0)
  def sum([head | tail]) do
    head + sum(tail)
  end
end
```

```
IO.puts(MyList.sum([1, 2, 3, 4, 5]))
# Извежда 15
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

или

Пример 2 за обхождане на списък

```
defmodule ListUtils do
  def print_elements([], do: :ok)
  def print_elements([head | tail]) do
    IO.puts(head)
    print_elements(tail)
  end
end
```

```
ListUtils.print_elements(["Apple", "Banana", "Cherry"])
```

Анализ на Пример 2

Функцията `print_elements([])` е *базов случай*, който спира рекурсията.

`print_elements([head | tail])` извежда текущия елемент (главата) и след това *рекурсивно* извиква функцията с остатъка от списъка (опашката).

2.2. Използване на Enum модул

Elixir предоставя модул `Enum`, който включва много полезни функции за работа с колекции. Тези функции действат на принципа на функционалния стил на работа и предоставят итерационни функции.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

1) Примери за Enum.each

Обхождане на списък с Enum.each

```
Enum.each([1, 2, 3, 4, 5], fn x -> IO.puts(x) end)
```

2) Приложение на Enum.map

```
squared = Enum.map([1, 2, 3, 4, 5], fn x -> x * x end)
```

```
IO.inspect(squared)
```

```
# Извежда [1, 4, 9, 16, 25]
```

3) Приложение на Enum.reduce

```
sum = Enum.reduce([1, 2, 3, 4, 5], 0, fn x, acc -> acc + x end)
```

```
IO.puts(sum)
```

```
# Извежда 15
```

3. “Reduce” и “map” алгоритми

map и reduce са две основни операции, които работят с колекции.

map. map е функция, която трансформира всеки елемент от колекция (например списък), като прилага определена функция върху всеки от тях.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

1) Пример за map

```
numbers = [1, 2, 3, 4, 5]
squared = Enum.map(numbers, fn x -> x * x end)
IO.inspect(squared)
# Извежда [1, 4, 9, 16, 25]
```

В случая Enum.map/2 взема списък от числа и прилага анонимна функция, която събира квадратите на числата.

reduce. reduce е функция, която се използва за "събиране" на елементи от колекция в една стойност, използвайки точно определена операция.

2) Пример за reduce

```
numbers = [1, 2, 3, 4, 5]
sum = Enum.reduce(numbers, 0, fn x, acc -> x + acc end)
IO.puts(sum)
# Извежда 15
```

Във вторият пример Enum.reduce/3 взема списък от числа и започва с акумулатор (в случая 0). Функцията, подадена на reduce, добавя текущия елемент x към акумулатора acc.

3) Итерационна структура for

В Elixir можете да използвате конструкцията for, но тя не е цикъл в смисъл на традиционните императивни езици. Вместо това, for в Elixir е предназначена за генериране на нови колекции. Тя работи подобно на map и връща нов списък, който съдържа елементите, генерирани от блоковия код.

В Elixir се използва знакът <- е известен като оператор за извличане (или "оператор за обхождане") и се използва основно в контекста на конструкцията for, както и в pattern matching (съвпадение на шаблони) при работа с итерации и списъци.

1) Употреба на <-

1.1.) В for конструкции:

Знакът <- се **използва за извличане на елементи от колекция**, като позволява на програмата да обхожда всеки елемент от списък или друга колекция.

Пример:

```
numbers = [1, 2, 3, 4, 5]
squared = for x <- numbers, do: x * x
IO.inspect(squared)
# Извежда [1, 4, 9, 16, 25]
```

В този случай, `x <- numbers` казва на Elixir да итерира през всеки елемент в списъка `numbers` и за всеки елемент да присвоява стойност на `x`. После, за всяко `x`, блокът `do: x*x` изчислява квадрата и резултатът се добавя в новия списък `squared`.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Различия между двата метода – синтаксис, изразителност и изпълнение

Пример 1: Списъчно разширение (list comprehension)

```
numbers = [1, 2, 3, 4, 5]
squared = for x <- numbers, do: x * x
IO.inspect(squared) # Извежда [1, 4, 9, 16, 25]
```

Пример 2: Използване на Enum.map/2

```
numbers = [1, 2, 3, 4, 5]
squared = Enum.map(numbers, fn x -> x * x end)
IO.inspect(squared) # Извежда [1, 4, 9, 16, 25]
```

И двата примера, пресмятат квадратите на числата в масива numbers, но използват различни подходи – списъчно разширение (list comprehension) и функцията Enum.map/2. Нека разгледаме разликите между тях, както и предимствата и недостатъците на всеки от подходите:

Синтаксис:

- Списъчното разширение е по-лаконично и често се смята за по-четимо, когато операциите са прости и непосредствени.
- Enum.map изисква да се напише анонимна функция, което може да направи кода малко по-дълъг в случаите на прости операции, но позволява по-голяма гъвкавост при работа с по-сложни функции.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Изразителност:

- Списъчното разширение е по-лесно за разширяване с условия, тъй като можете да добавите `when` към структурата на `for`, за да филтрирате стойности.
- `Enum.map` предоставя множество по-широки функции от `Enum` модула, като `Enum.filter`, `Enum.reduce` и т.н., което предоставя мощни инструменти за манипулиране на колекции.

Изпълнение:

- Няма значителна разлика в производителността между двата метода за малки колекции, но при работа с много големи списъци или сложни операции, използването на `Enum` функции може да е по-оптицизирано.

Препоръки за избор на метод

Използвайте списъчно разширение, когато:

- Операцията е проста и лесна за разбиране.
- Искате да добавите условия за филтриране на стойностите в същия контекст.

Използвайте `Enum.map`, когато:

- Работите със сложни функции, които могат да усложнят изразите в списъчното разширение.
- Искате да използвате функции от модула `Enum`, които добавят допълнителна гъвкавост и функционалност.

Обобщение

И двата подхода имат своите предимства и недостатъци. Кой да изберете зависи от контекста на вашия код, от сложността на операциите и от вашите лични предпочитания за четимост и организация на кода.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

1.2.) В съвпадение на шаблони:

Операторът за извличане на елемент <- също може да се използва в контекста на list comprehensions (разбирания) и приемането на елементи от колекции при pattern matching. Например, ако искате да вземете всеки елемент и да направите нещо с него, можете да използвате <- в for или Enum.each:

Пример:

```
list_of_tuples = [{1, "one"}, {2, "two"}, {3, "three"}]
```

```
for {num, word} <- list_of_tuples do  
  IO.puts("#{num} is #{word}")  
end
```

В примера всяка двойка от list_of_tuples се разпада на num и word.

Обобщение

В Elixir не се използват цикли while и for в традиционния смисъл, а вместо да пишете цикли, можете да разчитате на рекурсия и функционални конструкции, предоставени от модула Enum. Циклите и итерациите в Elixir следват функционалния стил на програмиране и предлагат мощни методи за работа с данни, което е в съответствие с езиковата философия на Elixir. Операторът <- в Elixir е мощен инструмент за работа с колекции, позволяващ ви да изразявате итерации и шаблонни съвпадения по чист и четим начин. Той е ключова част от функционалния и декларативен стил на програмиране, характерен за Elixir.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

4. Enumerables

В Elixir, **Enumerable** е протокол, който се използва за работа с колекции от данни. Той предоставя интерфейс за колекции като списъци, множества, мапове и др., които могат да бъдат обходени и преобразувани. Чрез Enumerable можете да използвате функции като `map`, `reduce`, `filter`, и много други операции.

Пример за използване на Enumerable

Използвайки списък, можем да видим как работят методите от модула `Enum`, който имплементира протокола `Enumerable`.

Примерен списък

```
numbers = [1, 2, 3, 4, 5]
```

Пример за map

```
squared_numbers = Enum.map(numbers, fn x -> x * x end)
IO.inspect(squared_numbers)
# Извежда [1, 4, 9, 16, 25]
```

Пример за reduce

```
sum = Enum.reduce(numbers, 0, fn x, acc -> x + acc end)
IO.puts(sum)
# Извежда 15
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

5. Eager vs Lazy

В Elixir, концепцията за eager (нетърпеливи) и lazy (бавни) оценки е важна, особено когато работим с колекции чрез модула Enum и Stream.

Eager evaluation: Прилаганите функции веднага обработват целия набор от данни още в момента, в който се извикат. Например, Enum.map моментално обработва всички елементи от списък и връща нова колекция.

Lazy evaluation: Функции от модула Stream обработват елементи един по един, само когато е необходимо. Това е полезно за работа с големи колекции, тъй като се намалява използваната памет и се увеличава производителността.

Пример за eager и lazy

Eager evaluation с Enum

```
eager_result = Enum.map(1..5, fn x -> x * x end)
```

```
IO.inspect(eager_result)
```

Извежда [1, 4, 9, 16, 25]

Lazy evaluation с Stream

```
lazy_result = Stream.map(1..5, fn x -> x * x end)
```

Едва при извикване, например с Enum.to_list, обработваме стойностите

```
result_list = Enum.to_list(lazy_result)
```

```
IO.inspect(result_list)
```

Извежда [1, 4, 9, 16, 25]

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

6. Pipe оператор

Pipe операторът (`|>`) е един от най-важните и мощни оператори в Elixir, който позволява да се предават резултати от една функция в друга. Той помага за подобряване на четимостта на кода и за по-лесно следене на потока на данни.

Пример за Pipe оператор

Използване на Pipe оператор

result =

1..5

|> Enum.map(fn x -> x * x end)

|> Enum.filter(fn x -> rem(x, 2) == 0 end)

IO.inspect(result) # Извежда [4, 16]

Анализ

Започваме с диапазон от числа от 1 до 5. Първо, използваме Enum.map, за да извършим повдигането на степен на всяко число. След това, с Enum.filter, филтрираме само четните числа.

Обобщение

Използването на Enumerables, разликата между eager и lazy оценки, и установяването как работи pipe операторът е от решаващо значение за програмирането на Elixir. Тези концепции не само че помагат в написването на по-четим и подреден код, но и подобряват производителността и работа с данни. Чрез тях можете да реализирате сложна логика по начин, който е едновременно ефективен и елегантен.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

7. Stream

Функции за създаване и композиране на потоци.

Потоците могат да се композират, като Lazy Enum. Всеки Enum, който генерира елементи един по един по време на изброяване, се нарича поток. Например, диапазонът на Elixir е поток:

```
range = 1..5
```

```
1..5
```

```
Enum.map(range, &(&1 * 2))
```

```
[2, 4, 6, 8, 10]
```

В горния пример, докато картографираме диапазона, елементите, които се изброяват, бяха създадени един по един, по време на изброяване. Модулът Stream позволи да картографираме обхвата, без да задействаме изброяването му:

```
range = 1..3
```

```
stream = Stream.map(range, &(&1 * 2))
```

```
Enum.map(stream, &(&1 + 1))
```

```
[3, 5, 7]
```

Обърнете внимание, че започнахме с диапазон и след това създадохме поток, който има за цел да умножи всеки елемент в диапазона с 2. В този момент не беше направено изчисление. Само когато се извика Enum.map/2, ние всъщност изброихме всеки елемент в диапазона, умножаваме го по 2 и добавяме 1. Казваме, че функциите в Stream са мързеливи, а функциите в Enum са нетърпеливи.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Тъй като са мързеливи потоците са полезни при работа с големи (или дори безкрайни) колекции. При свързването на много операции с Enum се създават междинни списъци, докато Stream създава рецепта за изчисления, които се изпълняват в по-късен момент. Нека видим друг пример:

1..3

```
|> Enum.map(&IO.inspect(&1))
```

```
|> Enum.map(&(&1 * 2))
```

```
|> Enum.map(&IO.inspect(&1))
```

1

2

3

2

4

6

```
#=> [2, 4, 6]
```

Обърнете внимание, че първо отпечатахме всеки елемент в списъка, след това умножихме всеки елемент по 2 и накрая отпечатахме всяка нова стойност. В този пример списъкът се преброява три пъти. Нека да видим пример с потоци:



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
stream = 1..3
|> Stream.map(&IO.inspect(&1))
|> Stream.map(&(&1 * 2))
|> Stream.map(&IO.inspect(&1))
Enum.to_list(stream)
1
2
2
4
3
6
#=> [2, 4, 6]
```

Въпреки че крайният резултат е същият, редът, в който са отпечатани елементите, се промени! С потоци отпечатваме първия елемент и след това отпечатваме неговия двоен. В този пример списъкът беше изброен само веднъж! Това имаме предвид, когато казахме по-рано, че потоците са **съставни, мързеливи изброими**. Обърнете внимание, че можем да извикаме `Stream.map/2` няколко пъти, като ефективно съставяме потоците и ги държим мързеливи. Изчисленията се извършват само когато извикате функция от модула `Enum`.

Подобно на `Enum`, функциите в този модул работят в линейно време. Това означава, че времето, необходимо за извършване на операция, нараства със същата скорост като дължината на списъка. Това се очаква при операции като `Stream.map/2`. В края на краищата, ако искаме да прекосим всеки елемент на поток, колкото по-дълъг е потокът, толкова повече елементи трябва да прекосим и толкова повече време ще отнеме.

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Създаване на потоци (Streams)

В стандартната библиотека на Elixir има много функции, които връщат потоци, някои примери са:

- **IO.stream/2** - предава входни линии, един по един
- **URI.query_decoder/1** - декодира низ от заявка (“чифт по двойка”)

Този модул също така предоставя много удобни функции за създаване на потоци, като **Stream.cycle/1**, **Stream.unfold/2**, **Stream.resource/3** и др.

Обърнете внимание, че функциите в този модул гарантирано връщат изброени числа. Тъй като изброимите могат да имат различни форми (структури, анонимни функции и т.н.), функциите в този модул могат да върнат всяка от тези форми и това може да се промени по всяко време. Например, функция, която днес връща анонимна функция, може да върне структура в бъдещи издания.

8. IO module

IO модулът е основният механизъм в Elixir за четене и запис на стандартен вход/изход (: **stdio**), стандартна грешка (: **stderr**), файлове и други IO устройства. Използването на модула е доста просто:

```
iex> IO.puts "hello world"
hello world
:ok
iex> IO.gets "yes or no? "
yes or no? yes
"yes\n"
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

По подразбиране функциите в IO модула четат от стандартния вход и записват в стандартния изход. Можем да променим това, като предадем например: `stderr` като аргумент (за да запишем на стандартното устройство за грешка):

```
iex> IO.puts :stderr, "hello world"
hello world
:ok
```

9. File module

Модулят `File` <https://docs/stable/elixir/File.html> __ съдържа функции, които ни позволяват да отваряме файлове като IO устройства. По подразбиране файловете се отварят в двоичен режим, което изисква разработчиците да използват специфичните функции `IO.binread/2` и `IO.binwrite/2` от IO модула:

```
iex> {:ok, file} = File.open "hello", [:write]
{:ok, #PID<0.47.0>}
iex> IO.binwrite file, "world"
:ok
iex> File.close file
:ok
iex> File.read "hello"
{:ok, "world"}
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Файл също може да бъде отворен с: utf8 кодиране, което казва на модула File да интерпретира байтовете, прочетени от файла като UTF-8-кодирани байтове.

Освен функции за отваряне, четене и писане на файлове, Файловият модул има много функции за работа с файловата система. Тези функции са кръстени на техните еквиваленти в UNIX. Например File.rm/1 може да се използва за премахване на файлове, File.mkdir/1 за създаване на директории, File.mkdir_p/1 за създаване на директории и цялата им родителска верига. Има дори File.cp_r/2 и File.rm_rf/2 за съответно копиране и премахване на файлове и директории рекурсивно (т.е. копиране и премахване на съдържанието на директориите).

Ще забележите също, че функциите в модула File имат два варианта: един „обикновен“ вариант и друг вариант, който има същото име като обикновената версия, но със завършващ знак (!). Например, когато четем файла "hello" в горния пример, използваме File.read/1. Като алтернатива можем да използваме File.read!/1:

```
iex> File.read "hello"
```

```
{:ok, "world"}
```

```
iex> File.read! "hello"
```

```
"world"
```

```
iex> File.read "unknown"
```

```
{:error, :enoent}
```

```
iex> File.read! "unknown"
```

```
** (File.Error) could not read file unknown: no such file or directory
```

Забележете, че когато файлът не съществува, версията с! повдига грешка. Версията без! е предпочитана, когато искаме да обработим различни резултати, като използваме съвпадение на шаблони:

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
case File.read(file) do
  {:ok, body}    -> # do something with the `body`
  {:error, reason} -> # handle the error caused by `reason`
end
```

Ако обаче очакваме файлът да е там, промяната на “!” е по-полезна, тъй като предизвиква смислено съобщение за грешка. Избягвайте да пишете:

```
{:ok, body} = File.read(file)
```

тъй като в случай на грешка `File.read/1` ще върне `{: error, reason}` и съвпадението на шаблона ще се провали. Все пак ще получите желания резултат (повдигната грешка), но съобщението ще бъде за модела, който не съвпада (като по този начин няма да се знае за какво всъщност е грешката).

Ако не искате да се справите с евентуална грешка (т.е. искате да се появи), за предпочитане е да използвате `File.read!/1`.

10. Path module

По-голямата част от функциите в модула `File` очакват пътища като аргументи. Най-често тези пътища ще бъдат обикновени двоични файлове. Модулът `Path` <https://docs.stable/elixir/Path.html> — предоставя възможности за работа с такива пътища:

```
iex> Path.join("foo", "bar")
```

```
"foo/bar,,
```

```
iex> Path.expand("~/hello")
```

```
"/Users/jose/hello"
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Използването на функции от модула Path вместо просто манипулиране на двоични файлове е за предпочитане, тъй като модулет Path се грижи прозрачно за различните операционни системи. Например, Path.join/2 се присъединява към път с наклонени черти (/) в Unix-подобни системи и с обратна наклонена черта (\) в Windows.

С това покрихме основните модули, които Elixir предоставя за работа с IO и взаимодействие с файловата система.

11. Процеси и ръководители на групи

Може би сте забелязали, че File.open/2 връща кортеж като `{:ok, pid}`:

```
iex> {:ok, file} = File.open "hello", [:write]
{:ok, #PID<0.47.0>}
```

Това се случва, защото IO модулет действително работи с процеси. Когато пишем IO.write(**pid**, **binary**), IO модулет ще изпрати съобщение до процеса, идентифициран от pid с желаната операция. Нека да видим какво ще стане, ако използваме наш собствен процес:

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
iex> pid = spawn fn ->
...> receive do: (msg -> IO.inspect msg)
...> end
#PID<0.57.0>
iex> IO.write(pid, "hello")
{:io_request, #PID<0.41.0>, #PID<0.57.0>, {:put_chars, :unicode, "hello"}}
** (ErlangError) erlang error: :terminated
```

След `IO.write/2` можем да видим разпечатана заявка, изпратена от `IO` модула (кортеж от четири елемента). Скоро след това виждаме, че се проваля, тъй като `IO` модул очаква някакъв резултат, който не сме предоставили.

Модулът `StringIO` <https://docs/stable/elixir/StringIO.html> __ осигурява реализация на съобщенията на `IO` устройство върху низовете:

```
iex> {:ok, pid} = StringIO.open("hello")
{:ok, #PID<0.43.0>}
iex> IO.read(pid, 2)
"he"
```

Чрез моделиране на `IO` устройства с процеси, виртуалната машина `Erlang` позволява на различни възли в една и съща мрежа да обменят файлови процеси, за да четат/записват файлове между възлите. От всички `IO` устройства има едно, което е специално за всеки процес: лидерът на групата.

Когато пишете на: `stdio`, всъщност изпращате съобщение до ръководителя на групата, който записва към дескриптора на стандартния входен файл:

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

```
iex> IO.puts :stdio, "hello"
```

```
hello
```

```
:ok
```

```
iex> IO.puts Process.group_leader, "hello"
```

```
hello
```

```
:ok
```

Лидерът на групата може да бъде конфигуриран за процес и се използва в различни ситуации. Например, при изпълнение на код в отдалечен терминал, той гарантира, че съобщенията в отдалечен възел се пренасочват и отпечатват в терминала, който е задействал заявката.

[12. iodata и chardata](#)

Във всички примери по-горе използвахме двоични файлове при писане във файлове. В главата „Двоични файлове, низове и символни списъци“ споменахме как низовете са просто байтове, докато символните списъци са списъци с кодови точки.

Функциите в IO и File също позволяват да се дават списъци като аргументи. Не само това, те също така позволяват да се дава смесен списък от списъци, цели числа и двоични файлове:

```
iex> IO.puts 'hello world'
```

```
hello world
```

```
:ok
```

```
iex> IO.puts ['hello', ?\s, "world"]
```

```
hello world
```

```
:ok
```


Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Това обаче изисква известно внимание. Списъкът може да представлява или куп байтове, или куп знаци и кой от тях да се използва зависи от кодирането на IO устройството. Ако файлът се отвори без кодиране, се очаква файлът да е в необработен режим и трябва да се използват функциите в IO модула, започващи с `bin*`. Тези функции очакват `iodata` като аргумент; те очакват да бъде даден списък от цели числа, представляващи байтове и двоични файлове.

От друга страна, `stdio` и файловете, отворени с: `utf8` кодиране, работят с останалите функции в IO модула. Тези функции очакват `char_data` като аргумент, т.е. списък от символи или низове.

Въпреки че това е тънка разлика, трябва само да се притесняваме за тези подробности, ако възнамеряваме да предаваме списъци на тези функции. Двоичните файлове вече са представени от основните байтове и като такива тяхното представяне винаги е необработено.

13. Работа с файлове

1. Работа с битови низове

Битови низове (или бинарни низове) в Elixir се представят с големи (<< ... >>) и могат да се манипулират по различни начини. Например, за да проверите дали един битов низ съдържа друг, можете да използвате функции за търсене.

```
# Дефиниране на битови низове
binary1 = <<1, 2, 3, 4, 5>>
binary_to_find = <<3, 4>>
# Функция за проверка за съвпадение
defmodule BinaryUtils do
  def contains?(binary, sub) do
    String.contains?(binary_to_string(binary), binary_to_string(sub))
  end
  # Помощна функция за конвертиране на битови низове в стрингове
  defp binary_to_string(binary) do
    binary |> :binary.bin_to_list() |> List.to_string()
  end
end
# Използване на функцията
BinaryUtils.contains?(binary1, binary_to_find)
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

2. Работа с двоични файлове

Двоичните файлове в Elixir се четат и записват чрез модулите File и IO. Например:

Четене на двоичен файл

Четене на двоичен файл

```
{:ok, binary_data} = File.read("path/to/your/binary/file")
```

Сега можете да работите с binary_data

Запис на двоичен файл

Записване на данни в двоичен файл

```
binary_data = <<1, 2, 3, 4, 5>>
```

```
File.write!("path/to/your/binary/file", binary_data)
```

3. Сравнение на двоични данни

За да сравните два двоични низа, можете да използвате ==, което ще проверява съдържанието на двата бита низове:

```
binary1 = <<1, 2, 3>>
```

```
binary2 = <<1, 2, 3>>
```

```
if binary1 == binary2 do
```

```
  IO.puts("Данните съвпадат")
```

```
else
```

```
  IO.puts("Данните не съвпадат")
```

```
end
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

4. Работа с текстови файлове

Модул: File

File.read/1: Зарежда съдържанието на текстов файл.

```
{:ok, content} = File.read("example.txt")
```

File.write/2: Записва съдържание в текстов файл.

```
File.write("example.txt", "Hello, World!")
```

File.read!: Чете файл и хвърля изключение, ако не може да бъде прочетен.

```
content = File.read!("example.txt")
```

File.write!/2: Записва съдържание в файл и хвърля изключение при грешка.

```
File.write!("example.txt", "Hello, World!")
```

File.stream!/1: Връща поток от редове от текстов файл.

```
File.stream!("example.txt") |> Enum.each(&IO.puts/1)
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

5. Работа с CSV файлове

Библиотека: NimbleCSV

NimbleCSV.define/2: Дефинира нов CSV парсер.

```
defmodule MyCSV do  
  use NimbleCSV, separator: ",", escape: "\""   
end
```

NimbleCSV.parse_string/1: Парсва CSV данни от стринг.

```
MyCSV.parse_string("a,b,c\nd,e,f")
```

NimbleCSV.parse_file/1: Парсва CSV файл.

```
MyCSV.parse_file("example.csv")
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

6. Работа с Excel файлове

Библиотека: Excel

Excel.read/1: Чете Excel файл.

```
{:ok, data} = Excel.read("example.xlsx")
```

Excel.write/2: Записва данни в нов Excel файл.

```
Excel.write("output.xlsx", data)
```

7. Работа с Word файлове

Библиотека: Docx

Docx.read/1: Чете съдържание от Word файл.

```
content = Docx.read("example.docx")
```

Docx.write/2: Записва текст в Word файл.

```
Docx.write("output.docx", content)
```

8. Работа с PDF файлове

Библиотеки: Hex.PDF, pdf

Pdf.read/1: Чете PDF файл и извлича текст.

```
{:ok, text} = Pdf.read("example.pdf")
```

Pdf.write/2: Записва PDF файл, но изисква ръчно построяване на съдържание.

```
Pdf.write("output.pdf", pdf_data)
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Работата с файлове в Elixir е гъвкава и мощна. Използвайте вградените функции на модула File за основни операции с текстови и двоични файлове, а за специфични формати като CSV, Excel, Word и PDF могат да се използват подходящи библиотеки.

Не забравяйте да добавите необходимите библиотеки в вашия проект с `mix.exs`, за да можете да ги използвате.

Пример:

```
defp deps do
  [
    {:nimble_csv, "~> 1.0"}, # за работа с CSV
    {:elixlsx, "~> 0.5"},    # за работа с Excel
    {:docx, "~> 0.1"},      # за работа с Word
    {:pdf, "~> 2.0"}        # за работа с PDF
  ]
End
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

Основната работа с файлове в Elixir, включва:

1. Създаване на файл и запис на данни.
2. Четене на данни от файл.
3. Добавяне на нови данни.
4. Редактиране/презаписване на данни.
5. Изтриване на файл.

Тези операции са основни за работа с текстови файлове в Elixir и предлагат много възможности за управление на данни в почти всяко приложение.

1. Създаване и записване на данни в текстов файл

```
defmodule FileHandler do
```

```
  @filename "data.txt"
```

```
  def create_file do
```

```
    # Създаване на файл и запис на начален текст
```

```
    File.write(@filename, "Това е стартов текст.\n")
```

```
  end
```

```
end
```

```
# Създаване на файла
```

```
FileHandler.create_file()
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

2. Четене на данни от текстов файл

```
defmodule FileHandler do
  # Предишните функции...

  def read_file do
    case File.read(@filename) do
      {:ok, content} ->
        IO.puts("Съдържание на файла:")
        IO.puts(content)

      {:error, reason} ->
        IO.puts("Неуспех при четене на файла: #{reason}")
    end
  end
end

# Четене на файла
FileHandler.read_file()
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

3. Добавяне на нови данни в текстов файл

```
defmodule FileHandler do
```

```
  # Предишните функции...
```

```
  def append_to_file(new_data) do
```

```
    File.write(@filename, new_data, [:append])
```

```
  end
```

```
end
```

```
# Добавяне на нов текст
```

```
FileHandler.append_to_file("Това е добавен текст.\n")
```

Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

4. Редактиране/презаписване на данни в текстов файл

Един начин за редактиране на файл е първо да го прочетем, после да поправим съдържанието и накрая да запишем отново.

```
defmodule FileHandler do
  # Предишните функции...

  def edit_file(old_content, new_content) do
    case File.read(@filename) do
      {:ok, content} ->
        new_content = String.replace(content, old_content, new_content)
        File.write(@filename, new_content)
      {:error, reason} ->
        IO.puts("Неуспех при четене на файла: #{reason}")
    end
  end
end

# Редактиране на файла
FileHandler.edit_file("Това е стартов текст.", "Текстът е редактиран.")
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

5. Изтриване на текстов файл

```
defmodule FileHandler do
```

```
  # Предишните функции...
```

```
  def delete_file do
```

```
    case File.rm(@filename) do
```

```
      :ok ->
```

```
        IO.puts("Файлът е изтрит успешно.")
```

```
      {:error, reason} ->
```

```
        IO.puts("Неуспех при изтриването на файла: #{reason}")
```

```
    end
```

```
  end
```

```
end
```

```
# Изтриване на файла
```

```
FileHandler.delete_file()
```



Тема 8. Рекурсивни и итеративни процеси при стартиране на функциите. Потоци

6. Изпълнение на програмата

Кодът по-горе може да бъде разделен на модули и функции, които можем да извикваме последователно в iex или в самостоятелен Elixir файл.

Например:

Създайте файл, назован file_handler.exs. След това можете да изпълните файла в терминала с:

```
elixir file_handler.exs
```

Или, можете да влезете в iex и да заредите модула, след което да извикате съответните функции.

